

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов развёртывания WAF, написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов на базе обратного прокси-сервера Nginx.

Ход работы

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)

Выполнить отправку вредоносного SQLi-запроса (например, `/?id=1' OR '1'=1`) напрямую к приложению (`http://127.0.0.1:5000`) с помощью `curl` и убедиться, что приложение его обрабатывает (возвращает код 200 OK). Повторить тот же SQLi-запрос через защищённый Nginx-периметр (`http://127.0.0.1`). Убедиться, что встроенный набор правил OWASP CRS перехватывает атаку, возвращая ошибку 403 Forbidden.

Задание 2. Написание пользовательских правил фильтрации (L7)

Выполнить отправку легитимного HTTP-запроса, подменив заголовок User-Agent на вредоносный сканер уязвимостей из индивидуального варианта, с помощью `curl -H "User-Agent: ..."`. Написать пользовательское правило WAF в подготовленном файле `nginx/custom.conf` с присвоением уникального ID из варианта. Правило должно блокировать запросы, содержащие данный User-Agent. Перезапустить контейнеры (`docker-compose restart waf`) и убедиться, что WAF успешно блокирует запрос сканера (код 403).

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

Выполнить стресс-тест целевого эндпоинта (согласно варианту) утилитой `Apache Bench`, отправив 500 запросов в 10 потоков: `ab -n 500 -c 10 http://127.0.0.1<ваш_эндпоинт>`. Зафиксировать, что сервер успешно обработал все запросы без ошибок (Failed requests: 0). Сконфигурировать директивы ограничения интенсивности трафика (`limit_req_zone` и `limit_req`) в файле `nginx/user_nginx.conf`, используя параметры `rate` и `burst` из индивидуального варианта. Перезапустить контейнер WAF, повторить стресс-тест утилитой `ab` и зафиксировать блокировку избыточных запросов (сервер должен начать отдавать код 503 Service Unavailable для запросов, превышающих лимит).

Задание 4. Проектирование профилей защиты (Без написания кода)

Составить конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (`client_body_timeout`, `client_header_timeout`) и обосновать свой выбор. Спроектировать архитектурную схему применения CDN (Content Delivery Network) в связке с WAF для защиты от объёмных (L3/L4) DDoS-атак.

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт	Rate Limit: rate	Rate Limit: burst
10	curl/7.68.0	10010	/api/profile	20r/s	30

Для выполнения лабораторной работы потребовалось установить Docker Desktop: он разворачивает лабораторный стенд из двух контейнеров (WAF и backend) одной командой, и без него запустить ModSecurity на Windows невозможно, так как пришлось бы вручную компилировать Nginx, ModSecurity и OWASP CRS под Windows, что заняло бы несколько дней. Вместе с Docker понадобился WSL 2: он обеспечивает Linux-окружение, без которого

Docker Desktop просто не запустится, и вы бы упёрлись в ошибку виртуализации. Для нагрузочного теста потребовался `hey.exe`: он создаёт 500 запросов в 10 потоков, только так можно превысить лимит 20r/s и увидеть ответы 503, тогда как обычный `curl` в цикле даёт всего 2-3 запроса в секунду и Rate Limiting не сработает. А чтобы собрать `hey.exe`, пришлось установить Go: готовый бинарник не скачивался с GitHub, и без Go пришлось бы искать `hey` на других зеркалах или ставить полный Apache ради утилиты `ab`.

Задание 1. Базовая фильтрация OWASP CRS

Что делали: проверили, как OWASP CRS реагирует на SQL-инъекцию: сначала напрямую к бэкенду (обход WAF), затем через защищённый Nginx-периметр.

1.1 — напрямую к бэкенду (порт 5000)

```
curl.exe -i "http://127.0.0.1:5000/?id=1%27%20OR%20%271%27=%271"
```

1.2 — через WAF-периметр (порт 80)

```
curl.exe -i "http://127.0.0.1/?id=1%27%20OR%20%271%27=%271"
```

Что получили:

Запрос	Код	Значение
Напрямую (5000)	HTTP/1.1 200 OK + Backend OK	Бэкенд уязвим, обрабатывает SQLi без фильтрации
Через WAF (80)	HTTP/1.1 403 Forbidden	OWASP CRS распознал сигнатуру SQLi и заблокировал запрос

Вывод: базовый набор OWASP CRS (850 правил) автоматически блокирует SQLi-инъекции на периметре Nginx без каких-либо пользовательских правил.

Задание 2. Пользовательское правило WAF

Что делали: написали правило ModSecurity для блокировки запросов с заданным User-Agent.

Что ввели в `nginx/custom.conf`:

```
SecRule REQUEST_HEADERS:User-Agent "@contains curl/7.68.0" \
    "id:10010,phase:1,deny,status:403,msg:'Blocked Custom Scanner UA - Variant 10'"
```

Что вводили в терминал:

Проверка ДО правила - запрос проходил

```
curl.exe -i -H "User-Agent: curl/7.68.0" http://127.0.0.1/
```

Перезапуск WAF

```
docker compose restart waf
```

Проверка ПОСЛЕ - блокировка

```
curl.exe -i -H "User-Agent: curl/7.68.0" http://127.0.0.1/
```

Контроль - другой UA не блокируется

```
curl.exe -i -H "User-Agent: Mozilla/5.0" http://127.0.0.1/
```

Что получили:

Запрос	Код	Значение
До правила, UA curl/7.68.0	200 OK	Правило ещё не применялось
После правила, UA curl/7.68.0	403 Forbidden	Правило 10010 сработало
После правила, UA Mozilla/5.0	200 OK	Правило точечное

Лог WAF подтверждает:

```
[error] ModSecurity: Access denied with code 403 (phase 1).
```

```
[id "10010"] [msg "Blocked Custom Scanner UA - Variant 10"]
```

```
[file ".../custom.conf"]
```

Вывод: пользовательское правило ModSecurity с id:10010 блокирует запросы с UA curl/7.68.0, не затрагивая другие клиенты.

Задание 3. Rate Limiting

Что делали: запустили стресс-тест до и после настройки Rate Limiting.

Что ввели в nginx/user_nginx.conf:

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=20r/s;
```

```
server {  
    listen 80;  
    server_name localhost;  
  
    location /api/profile {  
        limit_req zone=mylimit burst=30 nodelay;  
        proxy_pass ${BACKEND};  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
  
    location / {  
        proxy_pass ${BACKEND};  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

Что вводили в терминал:

ДО Rate Limiting

```
.hey.exe -n 500 -c 10 -U "Mozilla/5.0" -H "Content-Type: application/json" http://127.0.0.1/api/profile
```

Перезапуск WAF

```
docker compose restart waf
```

ПОСЛЕ Rate Limiting

```
.hey.exe -n 500 -c 10 -U "Mozilla/5.0" -H "Content-Type: application/json" http://127.0.0.1/api/profile
```

Контрольный одиночный запрос

```
curl.exe -i http://127.0.0.1/api/profile
```

Что получили:

Тест	Распределение кодов
ДО (без лимита)	[200] 500 responses — все запросы успешны
ПОСЛЕ (с лимитом 20r/s, burst=30)	[200] 39 responses + [503] 461 responses — большая часть отбита
Одиночный запрос	200 ОК — укладывается в лимит

Лог WAF:

```
[error] limiting requests, excess: 30.900 by zone "mylimit", client: 172.18.0.1
```

Вывод: директивы `limit_req_zone` и `limit_req` с параметрами `rate=20r/s` и `burst=30` ограничивают частоту запросов к `/api/profile`. Запросы сверх лимита получают 503 Service Unavailable, одиночные легитимные запросы проходят без проблем.

Примечание: для стресс-теста использован UA Mozilla/5.0 и Content-Type application/json, поскольку дефолтный UA hey/0.0.1 и text/html для GET-запроса блокируются OWASP CRS по аномальному счёту (правила 920350 + 920420). Это подтверждает, что WAF защищает даже от нагрузочных утилит.

Задание 4. Проектирование профилей защиты

4.1. Защита от Slowloris

Конфигурация Nginx:

```
client_body_timeout 10s;
client_header_timeout 10s;
keepalive_timeout 15s;
send_timeout 10s;
```

Обоснование:

Директива	Значение	Обоснование
-----------	----------	-------------

client_header_timeout	10 с	Легитимный клиент отправляет заголовки за миллисекунды. 10 с - с запасом для медленных сетей, но недостаточно для Slowloris. Дефолт 60 с - слишком много.
client_body_timeout	10 с	То же для тела запроса. Защита от slow POST.
keepalive_timeout	15 с	Время удержания idle-соединения. Слишком большое значение позволяет атакующему держать тысячи соединений.
send_timeout	10 с	Разрывает соединение, если клиент не читает ответ. Защита от slow read.

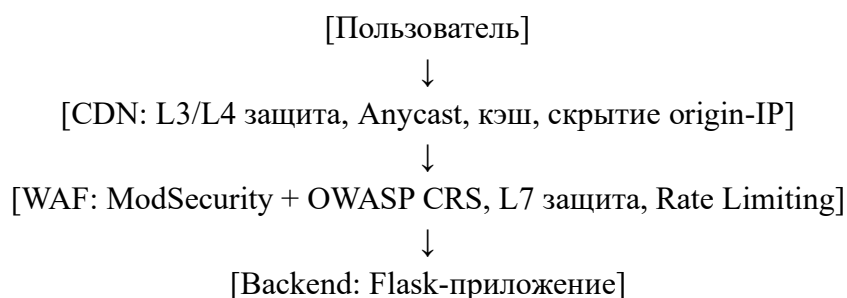
Принцип: Slowloris открывает множество соединений и отправляет заголовки по байту с паузами, исчерпывая пул worker_connections. Жёсткие таймауты (10 с) обрывают такие соединения.

Дополнительно:

limit_conn_zone \$binary_remote_addr zone=addr:10m;
limit_conn addr 20; не более 20 соединений с одного IP
limit_rate 500k; ограничение скорости отдачи

4.2. Эшелонированная защита CDN + WAF + Backend

Схема обработки трафика:



Роли слоёв:

1. CDN - распределённая сеть edge-серверов:

- Anycast-маршрутизация - трафик атаки «размазывается» по сотням точек, ни один узел не перегружается
- Кэширование статики - снижает нагрузку на origin
- Фильтрация L3/L4 - SYN-flood, UDP-flood, amplification гасятся на периферии
- Скрытие реального IP сервера приложения

2. WAF - анализ L7-трафика:

- Сигнатурный анализ OWASP CRS (SQLi, XSS, RFI, LFI, RCE)
- Пользовательские правила (блокировка специфичных UA)
- Rate Limiting (защита от L7 DDoS и брутфорса)
- Виртуальный патчинг уязвимостей

3. Backend - конечная точка, получает только валидные HTTP-запросы.

Ключевой принцип - defense in depth: CDN берёт на себя объём и L3/L4, WAF - сигнатуры и логику L7. Если атакующий пробьёт один слой, второй его остановит.

Вывод: получила практическое освоение методов развёртывания WAF, написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов на базе обратного прокси-сервера Nginx.